

Prototype | Create new objects by copying a prototypical instance — no class reference, no constructor call.

 DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

 DIVE 02 · 1 km
Train window
Theses, examples, terms

→

 DIVE 03 · 100 m
Park walk
How the pieces relate

→

 DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

Prototype builds new objects by copying an existing one — instead of newing a class, so creation needs no concrete class reference.

— THE CONTOURS

• Clone, don't construct
copy a ready instance instead of calling new

• Creation decoupled from class
the client needs no concrete class or constructor

• Skip the costly setup
duplicate a configured object, then tweak the copy

— USE IT WHEN — GoF applicability

 **Classes set at runtime**
the objects to instantiate are specified at run time, e.g. loaded or composed

 **Avoid a parallel factory**
to dodge a Creator hierarchy that just mirrors the product hierarchy

 Creational family: Factory Method · Abstract Factory · Builder · Prototype · Singleton — Prototype creates by copying a live instance, where the others instantiate classes; it trades subclassing for cloning.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

Prototype
declares clone() — the interface for copying itself

ConcretePrototype
implements clone(), returning a copy of itself

Client
makes new objects by asking a prototype to clone()

— IN THE WILD

 Object.create / spread
JS clones objects & arrays from an existing one

 java.lang.Cloneable
clone() — the JDK's built-in prototype hook

 structuredClone()
deep-copies live objects in modern JS

 Prototypal inheritance
JS objects literally delegate to a prototype

<> Client → prototype.clone() → a new ConcretePrototype copied from the original instance.

GoF intent: “Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client
holds a prototype

→

prototype.clone()
copy request

→

shallow or deep?
copy semantics

→

✓ new object
a ready copy

— WHY IT FOLLOWS

→ Clone an instance ⇒ no constructor call and no concrete class name are needed

→ The copy carries state ⇒ the new object starts pre-configured, then you adjust it

→ Registry of prototypes ⇒ pick a template by key at runtime and add new ones live

Abstract Factory may store and clone prototypes; Composite & Decorator trees are good clone candidates; a prototype registry resembles a Factory keyed by instance.

— STRUCTURE · UML

Prototype

«interface»

+ clone(): Prototype

Each ConcretePrototype implements clone() to copy itself; the client builds new objects from a proto type without ever naming its class.

— WHAT IT BUYS YOU

✓ Adds products at runtime — register a new prototype and clone it — no new class

✓ Specifies objects by value — a new “class” is just a differently-configured instance

✓ Reduces subclassing — cloning replaces a parallel Creator hierarchy

✓ Configures dynamically — compose prototypes at runtime, clone them on demand

✓ Cuts costly init — copy an expensive-to-build object instead of rebuilding it

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.”
— GoF, “Design Patterns”, 1994

— THE CODE

```
interface Prototype { clone(): Prototype; }
class Shape implements Prototype {
  constructor(public pos = { x: 0, y: 0 }) {}
  clone(): Shape {
    // deep-copy the mutable state:
    return new Shape({ ...this.pos });
  }
}
const a = new Shape(); a.pos.x = 5;
const b = a.clone(); // independent copy
```

— COMPARE THE ALTERNATIVES

Property	Prototype	Factory M.	Abstract F.	Builder
Creates by copying	✓	✗	✗	✗
Needs no subclass	✓	✗	~	✓
Carries existing state	✓	✗	✗	~
Add types at runtime	✓	✗	✗	✗
Hides concrete class	✓	✓	✓	✓

Best fit → Prototype: clone configured instances · Factory Method: subclass picks one · Abstract Factory: families · Builder: stepwise build. (“~” = partial.)

— INTERVIEW PITFALLS

Q “Shallow vs deep clone?”
Shallow copies the top level and shares nested references; deep recursively copies them. Choosing the wrong one is the classic Prototype bug.

Q “Does clone() call the constructor?”
Not necessarily — Java's Object.clone() copies fields and skips ctor logic, so invariants set in the constr utor can be missed.

Q “When beats a factory?”
When new objects should start from a configured, expensive-to-build instance, or when types are registered at runtime.

Q “How do you clone in modern JS?”
structuredClone() for deep copies (handles cycles); spread / Object.assign for shallow; JSON round-trips are lossy.

— NUANCES & GOTCHAS

• Shallow vs deep — shared nested references are the #1 cloning bug; decide the copy depth explicitly

• clone() can skip the ctor — Java's Object.clone() copies fields and bypasses constructor logic, so invariants break

• Circular references — naive deep copy loops forever; structuredClone handles cycles, JSON does not

• Not everything copies — identity, sockets, handles and caches usually shouldn't be duplicated

• JSON copy is lossy — it drops functions, undefined, Date / Map / Set and class identity

• Registry beats new — pick and clone a configured template instead of rebuilding from scratch

— WATCH OUT

 **Anti-pattern:** Skip it when objects are cheap to build or rarely vary — a constructor or factory is clearer. Deep-cloning graphs with shared identity, sockets or cycles is error-prone; copy explicitly.

 **Modern take:** in JS use structuredClone() for deep copies and spread / Object.assign for shallow ones; an explicit clone() or copy constructor beats hidden magic when objects own resources.

• VasyI Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) **v1**